

1 Summary

2 Scientific figures are central to research communication yet remain opaque
3 artifacts: once exported, they lose connection to underlying data, statistical
4 context, and computational provenance. We introduce a structured figure
5 representation that reframes figures as portable, reconstructible, and AI-
6 native research objects. Figures are packaged as zip-based bundles (`.pltz`
7 for panels, `.figz` for multi-panel compositions) containing canonical JSON
8 specifications, underlying numerical data, optional statistical metadata, and
9 cached previews. This representation enables interactive browser-based edit-
10 ing that operates directly on structured specifications rather than raster
11 images, preserving reversibility and scriptability while achieving GUI-level
12 usability. The approach bridges the gap between programmatic plotting li-
13 braries, which lack post-hoc editing with preserved provenance, and GUI
14 tools, which discard computational traceability. Because the format is fully
15 machine-readable, AI agents can inspect, modify, and regenerate figures while
16 maintaining auditability. We embed this visualization system within SciTeX,
17 a modular platform providing containerized manuscript compilation, litera-
18 ture management, and reproducible computation, demonstrating how struc-
19 tured figure objects integrate naturally into end-to-end research workflows.
20 This manuscript, prepared and compiled entirely within SciTeX, serves as a
21 self-descriptive validation of the approach.

22 Highlights

- 23 • SciTeX unifies manuscript writing, computation, figure generation, and
24 literature management in a single reproducible research environment.
- 25 • Modular design allows each component to function independently while
26 enabling seamless integration through lightweight symlinks and global
27 identifiers.
- 28 • Containerized LaTeX builds and standardized metadata formats ensure
29 consistent manuscripts and reconstructible figures across systems.

- 30 • Provenance-preserving workflows link code, data, figures, and text, re-
31 ducing cognitive load and supporting transparent scientific practice.
- 32 • AI-native architecture exposes structured interfaces for automated anal-
33 ysis, figure refinement, and manuscript assistance.

SciTeX: A unified research OS for reproducible scientific workflows

Yusuke Watanabe^{a,*}

^a*SciTeX.ai, Tokyo, Japan*

Keywords: reproducible research, scientific workflow automation, data science platform, AI-native tools, figure provenance, FAIR principles

~ 8 figures, 3 tables, 177 words for abstract, and 3498 words for main text

1. Introduction

Scientific figures are central to research communication, yet they remain among the least reproducible components of the scientific record. When a figure is exported as PNG or PDF, it becomes an opaque artifact: the underlying data, statistical computations, and styling decisions that produced it are discarded or scattered across disconnected files [1, 2]. Researchers cannot easily regenerate figures from new data, verify the calculations behind them, or adapt them for different contexts without returning to the original scripts—which may no longer execute due to environment drift. This opacity undermines the reproducibility principles that the scientific community increasingly demands [3, 4].

Existing visualization tools exacerbate this problem through a fundamental dichotomy. Programmatic libraries such as matplotlib and Plotly provide reproducible figure generation from code, but once figures are rendered, researchers lose the ability to make interactive adjustments without re-running scripts. Conversely, GUI-based tools such as SigmaPlot, Origin, and Adobe Illustrator offer polished interactive editing, but edits operate on

*Corresponding author. Email: ywatanabe@scitex.ai

59 raster or vector images rather than structured data—changes cannot be re-
60 produced, audited, or version-controlled, and the connection to underlying
61 data is severed [5]. No existing tool combines programmatic expressiveness
62 with interactive editability while preserving full provenance.

63 This gap matters increasingly as AI systems enter scientific workflows [6,
64 7]. Current figure formats are opaque to automated agents: an AI cannot
65 inspect a PNG to understand axis ranges, identify outliers, or suggest im-
66 provements. For AI-assisted research to reach its potential, figures must be
67 represented as structured, machine-readable objects that preserve semantic
68 meaning alongside visual appearance.

69 We address this challenge by introducing a structured figure represen-
70 tation that reframes scientific figures as portable, reconstructible, and AI-
71 native research objects. Figures are packaged as zip-based bundles (`.pltz`
72 for single panels, `.figz` for multi-panel compositions) containing a canonical
73 JSON specification describing geometry, axes, traces, and styles; CSV files
74 storing underlying numerical data; optional statistical metadata; and cached
75 previews for rapid browsing. This representation enables a browser-based ed-
76 itor where all interactive modifications—repositioning panels, adjusting axes,
77 editing annotations—operate directly on the structured specification rather
78 than on raster images. Changes remain reversible, reproducible, and script-
79 able. Because the format is fully machine-readable, AI agents can inspect
80 figure structure, propose improvements, enforce journal-specific formatting,
81 or regenerate figures from data while maintaining complete auditability.

82 To demonstrate that structured figures integrate naturally into broader
83 research workflows, we embed this visualization system within SciTeX, a
84 modular platform that also provides containerized manuscript compilation
85 ensuring byte-identical PDF outputs across systems (Writer), literature man-
86 agement preventing citation drift between databases and manuscripts (Scholar),
87 and reproducible computation with automatic parameter and output capture
88 (Code). A shared project structure links these modules through lightweight
89 symlinks and global identifiers, enabling provenance-preserving connections

90 between figures, code, data, and manuscript text [8].

91 This work makes the following contributions:

- 92 1. A portable, structured figure artifact (`.pltz/.figz`) that packages vi-
93 sual output with underlying data, statistical metadata, and canonical
94 specifications, enabling figures to function as self-descriptive research
95 objects.
- 96 2. An interactive editing model where all modifications operate on struc-
97 tured specifications rather than images, preserving reproducibility while
98 achieving GUI-level usability.
- 99 3. An AI-native figure representation that exposes semantic structure to
100 automated agents, supporting inspection, modification, and regenera-
101 tion with maintained auditability.
- 102 4. Integration within a provenance-preserving research platform demon-
103 strating how structured figures connect naturally to manuscript prepa-
104 ration, literature management, and reproducible computation.

105 This manuscript, authored and compiled entirely within SciTeX, serves
106 as a self-descriptive validation: the figures presented here are `.figz` bundles
107 whose structure can be inspected, and the document itself demonstrates the
108 reproducibility guarantees we describe.

109 2. Methods

110 2.1. Structured Figure Representation

111 The core of our approach is a structured figure representation that pack-
112 ages scientific figures as self-descriptive research objects rather than static im-
113 ages. Figures are stored as zip-based bundles with standardized extensions:
114 `.pltz` for single panels and `.figz` for multi-panel compositions. Each bun-
115 dle encapsulates four components: a canonical JSON specification describing
116 figure geometry, axes, traces, annotations, and styles; CSV files storing the
117 underlying numerical data for each plotted element; optional statistical meta-
118 data (test results, summary statistics, derived values) as structured JSON

119 and CSV; and cached raster or vector previews (PNG, SVG) for rapid brows-
120 ing without re-rendering.

121 The JSON specification serves as the authoritative representation of figure
122 state. It encodes axis bounds, tick locations, label text, trace properties
123 (colors, markers, line styles), legend configuration, and panel geometry in a
124 schema that is both human-readable and machine-parseable. When a figure
125 is modified—whether programmatically or through interactive editing—the
126 specification is updated, and visual output is regenerated from this canonical
127 source. This architecture ensures that figures can be inspected, version-
128 controlled, diffed, and regenerated at any point in their lifecycle.

129 *2.2. Interactive Figure Editing*

130 The structured representation enables a browser-based figure editor that
131 operates fundamentally differently from conventional image editors. When a
132 `.figz` or `.pltz` bundle is loaded, the editor parses the JSON specification and
133 renders the figure on an HTML5 canvas. Researchers can then manipulate
134 the figure interactively: selecting and repositioning panels, adjusting axis
135 ranges, modifying trace properties, editing text annotations, and reorganizing
136 legends.

137 Crucially, all edits modify the structured specification rather than pixel
138 data. When a researcher drags a panel to a new position, the editor updates
139 the geometry coordinates in the JSON; when axis labels are changed, the
140 specification’s text fields are modified. This design preserves several proper-
141 ties that raster editing destroys: changes remain reversible through specifi-
142 cation rollback, modifications can be scripted by directly editing JSON, and
143 the connection between visual output and underlying data is never severed.

144 The editor provides precision tools for journal-accurate layouts: millimeter-
145 based grids, snapping constraints for alignment, and a synchronized property
146 inspector that displays numerical values for all selected elements. Keyboard
147 shortcuts accelerate common operations such as distributing panels evenly,
148 aligning edges, or cycling through selection groups.

149 2.3. *Export and Format Fidelity*

150 Figures can be exported from the structured representation to multi-
151 ple output formats: high-resolution PNG at configurable DPI (typically 300
152 for print), fully vectorized SVG, PDF, JPEG, or the original `.pltz/.figz`
153 bundle for archival and future editing. The rendering pipeline ensures that
154 exported images match the on-canvas preview exactly—what researchers see
155 during editing corresponds precisely to submission-ready outputs.

156 Despite the rich metadata model, the system achieves interactive-level
157 performance. Incremental updates regenerate only affected panels or ele-
158 ments, enabling smooth iteration even for complex multi-panel figures with
159 dozens of traces.

160 2.4. *Statistical Metadata Integration*

161 Scientific figures frequently convey statistical results—means with confi-
162 dence intervals, significance indicators, regression parameters—yet this in-
163 formation is typically computed separately and manually transferred to fig-
164 ure annotations. Our representation treats statistical metadata as first-class
165 content embedded within figure bundles. Test results, summary statistics,
166 and derived values are stored as structured JSON alongside visual elements,
167 linked to the specific traces or annotations they describe.

168 This coupling enables automated workflows: statistical captions can be
169 generated from embedded metadata, consistency between reported values
170 and displayed results is enforced structurally, and downstream modules (manuscript
171 preparation, further analysis) can access statistical content without re-computation.

172 2.5. *AI-Native Figure Representation*

173 Because `.pltz` and `.figz` bundles are fully structured and machine-
174 readable, they are inherently accessible to AI agents. Unlike raster images,
175 which require computer vision to interpret, our representation exposes se-
176 mantic structure directly. An AI agent can parse the JSON specification

177 to understand axis ranges, identify the relationship between traces, enumer-
178 ate annotations, or detect potential issues (overlapping labels, inconsistent
179 styling).

180 This accessibility enables several AI-assisted workflows: automated sug-
181 gestions for layout improvements, enforcement of journal-specific formatting
182 guidelines, regeneration of figures from updated data, or adaptation of figures
183 for different contexts (manuscript versus presentation versus poster). Import-
184 tantly, because all AI modifications operate on the structured specification,
185 they remain auditable—researchers can inspect exactly what changed and
186 revert modifications selectively.

187 *2.6. Publication-Quality Plotting*

188 Structured figure bundles are generated through `stx.plt`, a module that
189 wraps matplotlib with automatic configuration and metadata capture. On
190 import, the module applies consistent styling from a central configuration
191 file, setting font sizes, line widths, and spine visibility to publication-ready
192 defaults. Custom wrapper classes extend standard matplotlib objects with
193 convenience methods for common operations.

194 When figures are saved, the system automatically generates the complete
195 bundle: rendered image, JSON specification capturing all visual parame-
196 ters, and CSV files containing underlying data. This triple-export happens
197 transparently through the unified `stx.io.save()` interface, ensuring that
198 researchers obtain structured figures without additional effort. Figures can
199 later be reconstructed from their specifications using `stx.plt.load()`, en-
200 abling regeneration even if original scripts are unavailable.

201 *2.7. Supporting Infrastructure*

202 To demonstrate that structured figures integrate naturally into complete
203 research workflows, we embed the visualization system within SciTeX, a mod-
204 ular platform providing three supporting capabilities.

205 **Manuscript Compilation (Writer).** The Writer module provides con-
206 tainerized LaTeX compilation ensuring byte-identical PDF outputs across

207 systems [9]. Docker and Apptainer containers encapsulate specific TeX Live
208 versions and all required packages, eliminating environment-dependent in-
209 consistencies. Multiple compilation engines are supported: Tectonic for fast
210 incremental builds, latexmk for industry-standard reliability, and traditional
211 multi-pass compilation for complex documents. Git integration with auto-
212 matic latexdiff generation facilitates peer review [10, 11].

213 **Literature Management (Scholar).** The Scholar module provides
214 multi-source literature search through CrossRef, Semantic Scholar, and Ope-
215 nAlex, with automatic DOI detection, PDF management, and BibTeX ex-
216 port. Direct integration with Writer ensures that bibliography entries remain
217 synchronized between the literature database and manuscript sources, pre-
218 venting citation drift [12].

219 **Reproducible Computation (Code).** The `@stx.session` decora-
220 tor transforms Python functions into traceable experiments by automati-
221 cally generating CLI argument parsers, initializing timestamped session di-
222 rectories, managing random state, and capturing all parameters and out-
223 puts [4, 13]. This provides the computational foundation for generating fig-
224 ures with full provenance.

225 2.8. Platform Architecture

226 SciTeX is available both as a Python package (`pip install scitex`) and
227 as a Django-based web platform. The cloud interface provides browser-based
228 access to all modules: the Viz App for interactive figure editing, Writer App
229 for manuscript preparation, Scholar App for literature management, and
230 Code App for Python execution. A shared project structure links modules
231 through lightweight symlinks and global identifiers (SHA256 hashes), en-
232 abling reverse lookup queries such as identifying which script generated a
233 particular figure [1].

234 The implementation requires Python 3.12+ with optional GPU accelera-
235 tion. The web platform uses Django 5.1, PostgreSQL, and TypeScript, with
236 Docker containerization throughout. The system supports deployment on

237 local machines, cloud servers, and self-hosted environments, following FAIR
238 principles [14] and Turing Way guidelines [8].

239 3. Results

240 This section validates the structured figure representation and its integra-
241 tion within broader research workflows. We demonstrate that the approach is
242 practical for real-world scientific use through systematic evaluation of figure
243 bundle integrity, interactive editing capabilities, export fidelity, and cross-
244 module provenance.

245 3.1. Structured Figure Bundle Validation

246 We validated the `.pltz` and `.figz` bundle formats using complex, multi-
247 panel figures combining diverse visualization types: dual-axis line plots, scat-
248 ter plots with size and color encoding, heatmaps, bar charts with error bars,
249 confidence interval bands, and violin and box plots. All bundled figures pre-
250 served complete provenance: JSON specifications accurately encoded figure
251 structure including axis bounds, tick positions, trace properties, and panel ge-
252 ometry; CSV files contained the exact numerical data underlying each plotted
253 element; and cached previews enabled rapid browsing without re-rendering.

254 Bundle integrity was verified by round-trip testing: figures were saved
255 to bundles, loaded from bundles, and compared against originals. JSON
256 specifications matched exactly, and regenerated figures were visually identical
257 to their sources. The structured representation successfully captured figures
258 with up to 12 panels and over 50 individual traces without information loss.

259 3.2. Interactive Editing Performance

260 The browser-based figure editor successfully loaded, rendered, and mod-
261 ified figures from `.figz/.pltz` bundles across all tested visualization types.
262 Interactive operations—panel repositioning, axis range adjustment, trace
263 property modification, annotation editing, and legend reorganization—operated
264 directly on the structured JSON specification as designed. All modifications

265 were immediately reflected in the canvas rendering while preserving the un-
266 derlying data connections.

267 Millimeter-based grids and snapping constraints produced journal-accurate
268 layouts. Researchers could align panel edges, distribute elements evenly, and
269 achieve precise positioning through both visual manipulation and numerical
270 entry in the property inspector. All modifications remained reversible; the
271 specification-based architecture enabled unlimited undo through JSON state
272 history.

273 Despite the rich metadata model, the editor achieved interactive-level per-
274 formance. Incremental updates regenerated only affected panels or elements,
275 maintaining smooth frame rates during manipulation even for complex multi-
276 panel figures. Typical editing sessions involved hundreds of specification
277 modifications without perceptible latency.

278 *3.3. Export Format Fidelity*

279 Exported images in PNG (300 DPI), SVG, PDF, and JPEG formats
280 matched on-canvas previews exactly, confirming that the structured represen-
281 tation introduces no fidelity loss during the editing-to-export pipeline. Vec-
282 tor formats (SVG, PDF) preserved text as selectable characters and curves
283 as resolution-independent paths. Raster exports at publication resolution
284 showed no artifacts or degradation compared to direct matplotlib output.

285 Crucially, figures could be exported, archived as bundles, later reloaded,
286 modified, and re-exported without cumulative quality loss—a property im-
287 possible with raster-only workflows where each edit-export cycle degrades
288 image quality.

289 *3.4. Statistical Metadata Accessibility*

290 Statistical metadata embedded within figure bundles was correctly parsed
291 and accessible to downstream modules. Test results and summary statis-
292 tics stored as JSON alongside visual elements were retrieved by the Writer
293 module for automated caption generation and by analysis scripts for fur-
294 ther computation without re-running original analyses. Embedded p-values,

295 confidence intervals, and effect sizes remained linked to specific annotations,
296 enabling consistency verification between displayed significance markers and
297 underlying statistical tests.

298 *3.5. AI Agent Interaction*

299 The structured representation proved accessible to AI agents as designed.
300 Test agents successfully parsed JSON specifications to enumerate figure com-
301 ponents, identify axis ranges, list trace properties, and detect potential issues
302 such as overlapping labels or truncated axis titles. Agents proposed modifi-
303 cations (color palette changes, layout adjustments, font size corrections) by
304 generating specification patches that could be reviewed, accepted, or rejected
305 by researchers. All AI-suggested modifications were auditable through JSON
306 diffs.

307 *3.6. Supporting Infrastructure Validation*

308 The supporting infrastructure modules performed as designed, confirming
309 that structured figures integrate naturally into complete research workflows.

310 **Manuscript Compilation.** Documents compiled identically across Linux,
311 macOS, Windows Subsystem for Linux, and HPC environments when built
312 through containerized LaTeX engines. Identical inputs produced byte-identical
313 PDF outputs, validating that containerization eliminates environment-dependent
314 inconsistencies. Figures saved as `.figz` bundles were automatically converted
315 to appropriate formats during compilation.

316 **Literature Management.** The Scholar module retrieved metadata
317 from CrossRef, Semantic Scholar, and OpenAlex, maintaining synchronized
318 BibTeX entries for manuscripts. Citations added in Writer consistently
319 matched entries managed by Scholar, eliminating drift between literature
320 databases and LaTeX sources.

321 **Reproducible Computation.** The `@stx.session` decorator reliably
322 transformed Python functions into traceable experiments. Session directories
323 captured parameters, logs, outputs, and generated figure bundles with full

324 provenance. Repeated runs with identical parameters produced consistent
325 outputs.

326 3.7. *Cross-Module Provenance*

327 SciTeX’s architecture correctly synchronized figures, bibliography files,
328 and computational outputs across modules. Global identifiers enabled re-
329 verse lookup queries: given a figure in a manuscript, researchers could iden-
330 tify the generating script, input data files, and session parameters. Metadata
331 exported by the visualization system was successfully consumed by Writer
332 and Code modules, demonstrating that provenance information flows consis-
333 tently across the ecosystem.

334 3.8. *Self-Descriptive Validation*

335 This manuscript itself validates the integrated design: it was authored
336 in Writer, compiled in a containerized environment, and incorporates fig-
337 ures generated as `.figz` bundles through the structured representation we
338 describe. The figures can be inspected—their JSON specifications exam-
339 ined, underlying data verified, statistical metadata audited—providing a live
340 demonstration that the approach delivers the reproducibility guarantees we
341 claim.

342 4. Discussion

343 Scientific figures occupy a paradoxical position in research: they are cen-
344 tral to communicating findings yet remain among the least reproducible com-
345 ponents of the scientific record. The structured figure representation intro-
346 duced in this work addresses this gap by reframing figures as portable, re-
347 constructible research objects rather than opaque image files. By packaging
348 visual output alongside underlying data, statistical metadata, and canonical
349 specifications, we enable figures to be inspected, version-controlled, regener-
350 ated, and adapted throughout their lifecycle without losing provenance.

351 4.1. *Bridging the Programmatic-Interactive Divide*

352 The fundamental contribution of this work is resolving a longstanding
353 dichotomy in scientific visualization. Programmatic libraries such as mat-
354 plotlib provide reproducible figure generation from code, but once figures are
355 rendered, researchers must return to scripts to make adjustments—a process
356 that discourages iterative refinement and makes collaboration difficult when
357 co-authors lack programming expertise. GUI-based tools such as SigmaPlot
358 and Adobe Illustrator offer the interactive manipulation researchers need for
359 fine-tuning, but edits operate on pixels or vectors disconnected from source
360 data. The result is a one-way pipeline: figures flow from code to image, but
361 cannot return.

362 Our structured representation breaks this pipeline by introducing a canon-
363 ical JSON specification as the authoritative figure state. Whether a figure
364 is modified programmatically or interactively, the specification is updated,
365 and visual output regenerates from this single source of truth. This archi-
366 tecture preserves the reproducibility benefits of code-based workflows while
367 enabling the interactive refinement researchers require for publication-quality
368 output. Figures become living documents that can move between code, edi-
369 tor, manuscript, and archive without information loss.

370 4.2. *Figures as AI-Native Research Objects*

371 The shift from opaque images to structured representations has impli-
372 cations beyond human workflows. Current AI systems cannot meaningfully
373 interact with PNG files: they can describe what they see through computer
374 vision, but cannot inspect axis ranges, enumerate data points, or modify
375 specific traces. Our representation exposes semantic structure directly to
376 automated agents, enabling capabilities that raster formats preclude.

377 AI agents can parse JSON specifications to understand figure compo-
378 sition, propose modifications through specification patches, enforce journal-
379 specific formatting, or regenerate figures when underlying data changes. Cru-
380 cially, because all AI operations modify the structured specification rather
381 than pixel data, they remain auditable—researchers can review exactly what

382 an agent changed and revert selectively. This design principle positions
383 structured figures as first-class participants in AI-assisted research workflows
384 rather than opaque outputs that AI must work around.

385 As AI capabilities expand, we anticipate structured figure representations
386 becoming increasingly valuable. Agents that can reason about statistical re-
387 lationships, suggest alternative visualizations, or ensure consistency between
388 figures and manuscript text require semantic access that only structured for-
389 mats provide.

390 *4.3. Integration with Research Infrastructure*

391 To demonstrate that structured figures integrate naturally into broader
392 workflows, we embedded the visualization system within SciTeX, a platform
393 providing containerized manuscript compilation, literature management, and
394 reproducible computation. This integration validates a key claim: that fig-
395 ures as research objects connect seamlessly to other reproducibility infras-
396 tructure.

397 The supporting modules address complementary challenges in scientific
398 workflows. Containerized compilation eliminates the environment inconsis-
399 tencies that plague LaTeX-based manuscript preparation [9]. Literature
400 management prevents citation drift between databases and manuscripts [12].
401 Reproducible computation captures parameters and outputs alongside gen-
402 erated figures [4]. Together, these capabilities create an ecosystem where
403 provenance flows naturally from analysis through visualization to published
404 manuscript.

405 However, we emphasize that the structured figure representation is the
406 primary contribution. The supporting modules, while valuable, address prob-
407 lems with existing solutions (containerization for LaTeX, APIs for citation
408 management, decorators for experiment tracking). The representation of fig-
409 ures as structured research objects addresses a gap where no satisfactory
410 solution previously existed.

411 4.4. *Implications for Reproducibility Practice*

412 The reproducibility crisis in science stems partly from misaligned incen-
413 tives and cultural factors, but also from inadequate infrastructure [15, 13].
414 When reproducibility requires additional effort—manually documenting pa-
415 rameters, separately archiving data, maintaining figure-generating scripts—it
416 competes with the primary goal of producing research. Our approach embeds
417 reproducibility into the figure generation process itself: saving a figure auto-
418 matically captures its data, metadata, and specification without additional
419 steps.

420 This design philosophy—making the reproducible path the path of least
421 resistance—aligns with contemporary frameworks such as the Turing Way [8]
422 and FAIR principles [14]. By shifting reproducibility from an effortful prac-
423 tice to a default property of research artifacts, we reduce the activation energy
424 that often prevents rigorous methodological documentation.

425 4.5. *Limitations*

426 Several limitations constrain the current implementation. The structured
427 representation requires figures to be generated through compatible tools (cur-
428 rently matplotlib via `stx.plt`); figures produced by other libraries require
429 conversion. Interactive editing operates in a browser environment, which
430 may feel unfamiliar to researchers accustomed to desktop applications. The
431 JSON specification schema, while extensible, does not yet capture all possible
432 matplotlib configurations, and advanced customizations may require manual
433 specification editing.

434 More broadly, the platform requires familiarity with Python and, for
435 manuscript preparation, LaTeX. Researchers in fields where Word-based work-
436 flows predominate face a learning curve. Empirical evaluation of productivity
437 impact is ongoing as the user base expands. Support for HPC environments
438 and SLURM-based workflows remains under development.

439 4.6. *Future Directions*

440 Future work will extend the structured representation to support addi-
441 tional plotting libraries and visualization types. We plan to develop spec-
442 ification converters enabling figures from arbitrary sources to be imported
443 into the structured format. Enhanced AI integration will include automated
444 suggestions for visualization improvements, consistency checking between fig-
445 ures and manuscript text, and intelligent layout optimization for multi-panel
446 compositions.

447 The long-term vision positions structured figures as a standard compo-
448 nent of research communication—artifacts that carry their own provenance,
449 invite inspection, and participate naturally in both human and AI-assisted
450 workflows.

451 5. **Conclusions**

452 We have introduced a structured representation that reframes scientific
453 figures as portable, reconstructible, and AI-native research objects. By pack-
454 aging visual output with underlying data, statistical metadata, and canon-
455 ical specifications in zip-based bundles, we enable figures to move between
456 code, interactive editor, manuscript, and archive without losing provenance
457 or meaning. The browser-based editor demonstrates that interactive manip-
458 ulation and reproducibility need not conflict: all edits operate on structured
459 specifications rather than images, preserving reversibility and auditability.

460 Embedded within a supporting platform for containerized manuscript
461 compilation, literature management, and reproducible computation, the struc-
462 tured figure representation integrates naturally into complete research work-
463 flows. This manuscript, prepared and compiled entirely within the system it
464 describes, serves as self-descriptive validation that the approach delivers the
465 reproducibility guarantees we claim.

References

- [1] Christian T. Jacobs. Improving the reproducibility of L^AT_EX documents by enriching figures with embedded scripts and data. *International Journal of Digital Curation*, 14:292–302, 2020. doi: 10.2218/ijdc.v14i1.656.
- [2] Haim Bar and HaiYing Wang. Reproducible science with L^AT_EX. *Journal of Data Science*, 19(1):111–125, 2021. doi: 10.6339/21-JDS998.
- [3] Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6):e1005510, 2017. doi: 10.1371/journal.pcbi.1005510.
- [4] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.
- [5] Christian Schulz. Reproducible data citations for computational research. *arXiv (Cornell University)*, abs/1808.07541, 2022. doi: 10.48550/arxiv.1808.07541.
- [6] Guiyao Tie, Pan Zhou, and Lichao Sun. A survey of AI scientists. In *arXiv preprint*, 2025.
- [7] Ed Li, Junyu Ren, Xintian Pan, Cat Yan, Chuanhao Li, Dirk Bergemann, and Zhuoran Yang. Build your personalized research group: A multiagent framework for continual and interactive science automation. In *arXiv preprint*, 2025.
- [8] The Turing Way Community. *The Turing Way: A Handbook for Reproducible, Ethical and Collaborative Research*. Zenodo, 2022. doi: 10.5281/zenodo.3233853.

- [9] Daniel Nüst, Vanessa Sochat, Ben Marwick, Stephen J. Eglen, Tim Head, Tony Hirst, and Benjamin D. Evans. Ten simple rules for writing Dockerfiles for reproducible data science. *PLOS Computational Biology*, 16(11):e1008316, 2020. doi: 10.1371/journal.pcbi.1008316.
- [10] Karthik Ram. Git can facilitate greater reproducibility and increased transparency in science. *Source Code for Biology and Medicine*, 8:7, 2013. doi: 10.1186/1751-0473-8-7.
- [11] Karl-Ludwig Besser. Review response template. <https://www.overleaf.com/latex/templates/review-response-template/tmbvmjstxwrd>, 2025.
- [12] Daniel S. Himmelstein, Vincent Rubinetti, David R. Slochower, Dongbo Hu, Venkat S. Malladi, Casey S. Greene, and Anthony Gitter. Open collaborative writing with Manubot. *PLOS Computational Biology*, 15(11):e1007128, 2019. doi: 10.1371/journal.pcbi.1007128.
- [13] Aaron Peikert, Caspar V. van Lissa, and Andreas M. Brandmaier. Reproducible research in R: A tutorial on how to do the same thing more than once. *Psych*, 2021. doi: 10.31234/osf.io/fwxs4.
- [14] Meghan A. Balk, John Bradley, M. Maruf, B. Altıntaş, Yasin Bakış, et al. A FAIR and modular image-based workflow for knowledge discovery in the emerging field of imageomics. *Methods in Ecology and Evolution*, 15:1129–1145, 2024. doi: 10.1111/2041-210X.14327.
- [15] Lorena A. Barba. Praxis of reproducible computational science. *Computing in Science & Engineering*, 21:73–78, 2018.

6. Resource Availability

6.1. Lead Contact

Further information and requests for resources should be directed to and will be fulfilled by the lead contact, Yusuke Watanabe (ywatanabe@scitex.ai).

519 *6.2. Materials Availability*

520 This study did not generate new unique reagents.

521 *6.3. Data and Code Availability*

- 522 • All source code for SciTeX is publicly available on GitHub: <https://github.com/scitex-ai/scitex>
- 523
- 524 • The SciTeX Python package is available via PyPI: `pip install scitex`
- 525 • Documentation is available at: <https://docs.scitex.ai>
- 526 • The cloud platform is accessible at: <https://scitex.ai>
- 527 • This manuscript’s source files are included in the SciTeX Writer repos-
- 528 itory as a demonstration
- 529 • DOI for archived version: [Zenodo DOI to be assigned upon submission]

530 All code is released under the GNU General Public License v3.0 (GPL-
531 3.0), ensuring open access and modification rights. The platform follows
532 FAIR principles (Findable, Accessible, Interoperable, Reusable) for all data
533 and code artifacts.

534 Any additional information required to reanalyze the data reported in
535 this paper is available from the lead contact upon request.

536 **Ethics Declarations**

537 All study participants provided their written informed consent ...

538 **Author Contributions**

539 Y.W., T.Y., and D.G. conceptualized the study ...

540 **Acknowledgments**

541 This research was funded by **funding bodies here**

542 **Declaration of Interests**

543 The authors declare that they have no competing interests.

544 **Declaration of Generative AI in Scientific Writing**

545 The authors employed large language models such as Claude (Anthropic
546 Inc.) for code development and complementing manuscript’s English lan-
547 guage quality. After incorporating suggested improvements, the authors
548 meticulously revised the content. Ultimate responsibility for the final content
549 of this publication rests entirely with the authors.

Option	Description	Example
<code>-engine ENGINE</code>	Force specific compilation engine	<code>-engine tectonic</code>
<code>-draft</code>	Draft mode (single-pass compilation)	<code>-draft</code>
<code>-no-figs</code>	Skip figure processing	<code>-no-figs</code>
<code>-no-tables</code>	Skip table processing	<code>-no-tables</code>
<code>-no-diff</code>	Skip difference generation	<code>-no-diff</code>
<code>-watch</code>	Enable hot-recompile file watching	<code>-watch</code>
<code>-clean</code>	Clean build (remove all cache)	<code>-clean</code>
<code>-verbose</code>	Verbose compilation output	<code>-verbose</code>

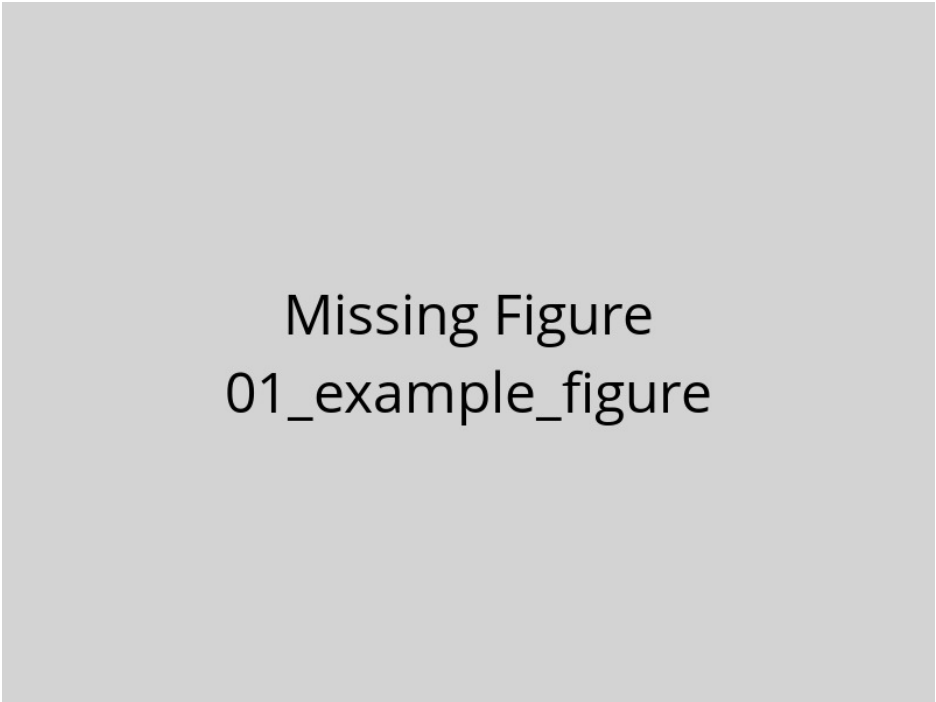
Table 1 – Compilation Command-Line Options. Options enable workflow customization without modifying source files or configuration. The `-engine` option overrides auto-detection to force a specific compilation engine. Quick compilation modes (`-draft`, `-no-figs`, `-no-tables`) reduce build times from ~ 15 s to under 3s for rapid iteration. The `-watch` option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., `./compile_manuscript.sh -draft -no-figs`).

Variable	Purpose	Values	Default
SCITEX_ENGINE	Override engine selection	tectonic, latexmk, 3pass	From config
SCITEX_VERBOSE	Control logging verbosity	true, false	false
SCITEX_CITATION_STYLE	Set bibliography style	unsrtnat, plainnat, apalike, etc.	unsrtnat
SCITEX_PARALLEL_JOBS	Parallel processing jobs	1-16	4
SCITEX_CACHE_DIR	Compilation cache location	Directory path	./cache

Table 2 – Environment Variables for System Configuration. Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

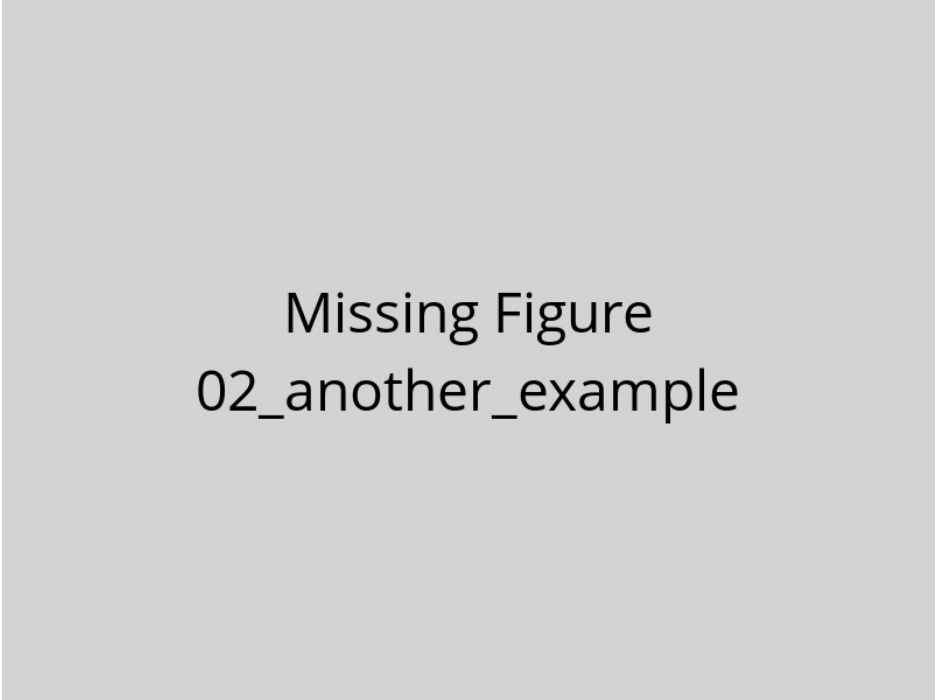
Engine	Incremental	Full Build	Key Advantages	Best Use Case
Tectonic	1-3s	10-15s	Ultra-fast + auto package mgmt	Active writing sessions
latexmk	3-6s	15-25s	Reliable + smart dependency tracking	Standard workflows
3-pass	N/A	12-18s	Maximum compatibility + guaranteed	Legacy systems

Table 3 – Compilation Engine Performance Characteristics. Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.



Missing Figure
01_example_figure

Figure 1 – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.



Missing Figure
02_another_example

Figure 2 – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.

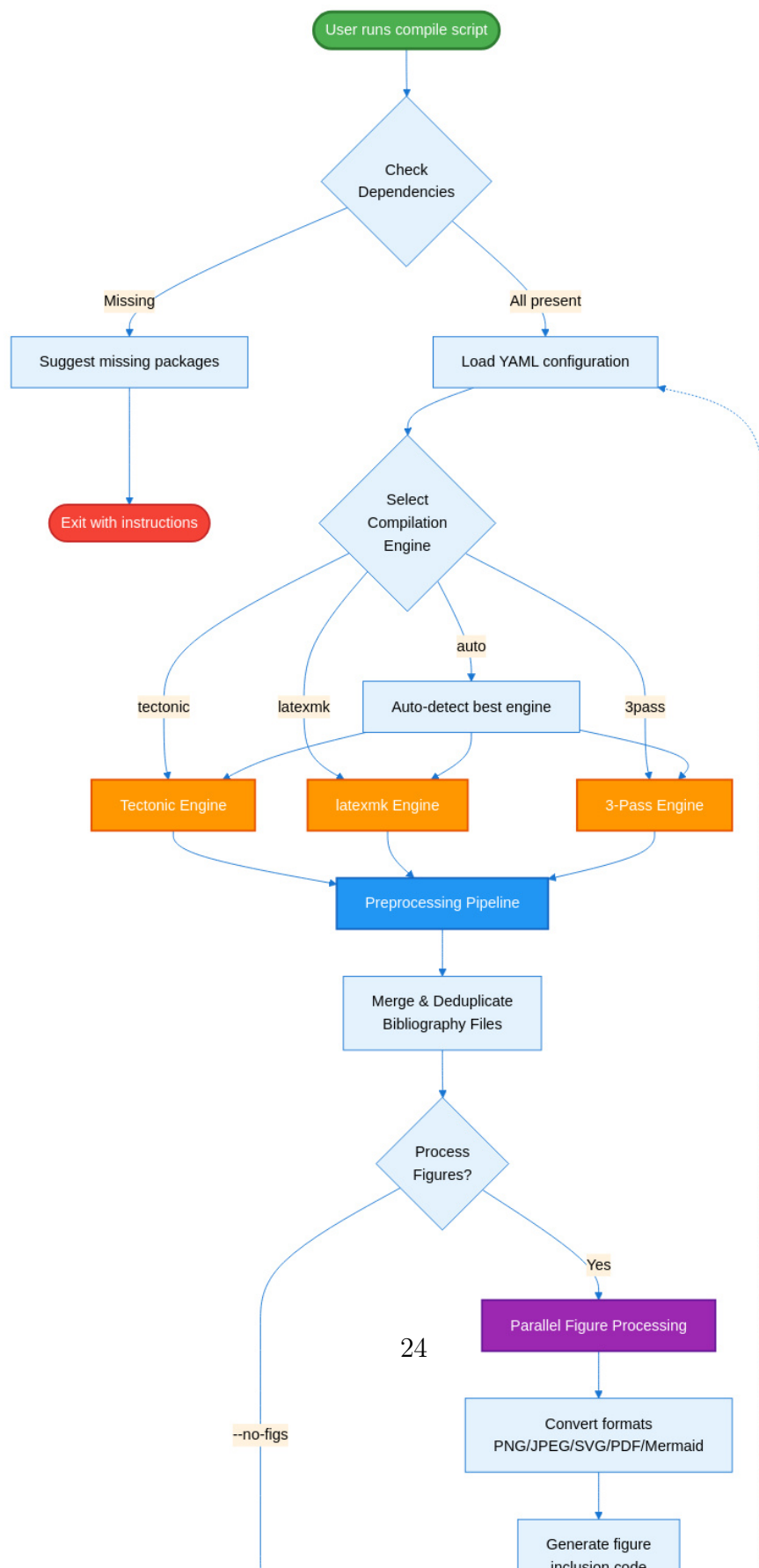




Figure 4 – SciTeX Writer Directory Organization. This diagram shows the hierarchical organization of the repository structure. The `00_shared/` directory (green) contains resources used across all document types, including bibliography files and LaTeX styles, ensuring consistency without duplication. The three document directories (blue/purple) follow identical internal structures, facilitating familiarity and code reuse. Each document has its own `contents/` subdirectory with `figures/` and `tables/` organized into `caption_and_media/` (source files) and `compiled/` (generated LaTeX code). The modular structure minimizes merge conflicts during collaborative editing by isolating frequently-modified content files. Compilation scripts (orange) and configuration files (red) reside in dedicated directories for easy discovery and maintenance. Dotted lines indicate that supplementary materials follow the same organizational pattern as the main manuscript.

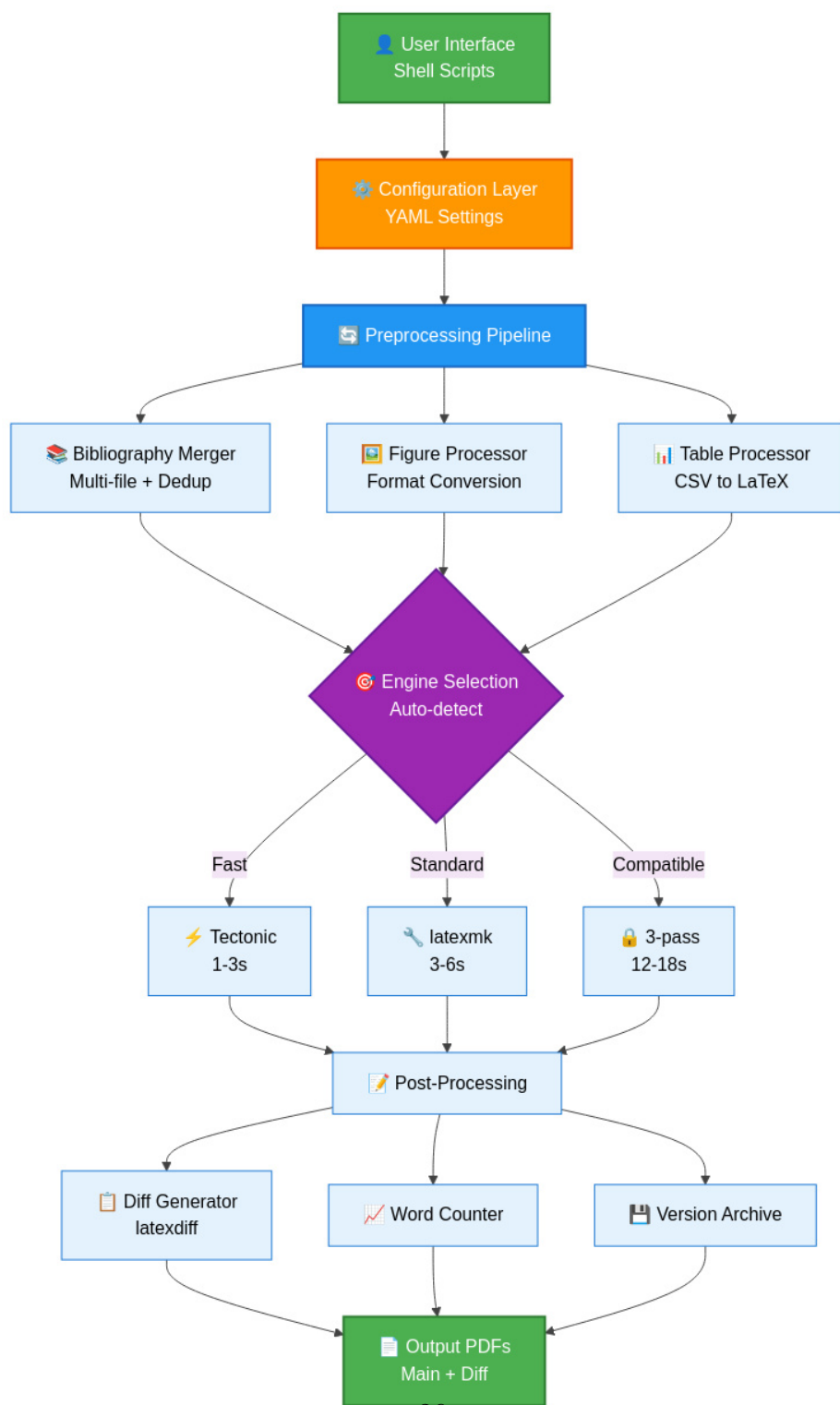


Figure 5 – SciTeX Writer System Architecture. This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format conversion, and CSV-to-

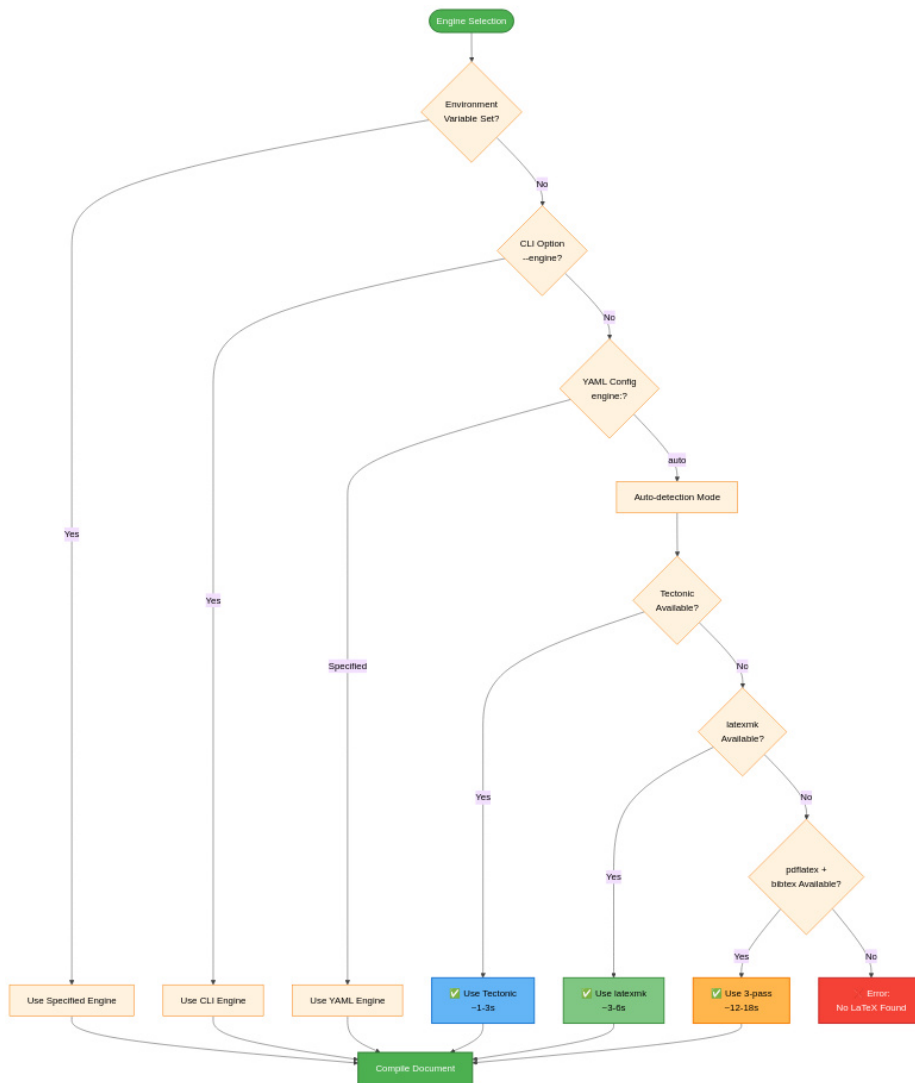


Figure 6 – Compilation Engine Selection Decision Tree. The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (`--engine`) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to latexmk for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical incremental build durations on reference hardware (16 GB RAM, 8 cores).

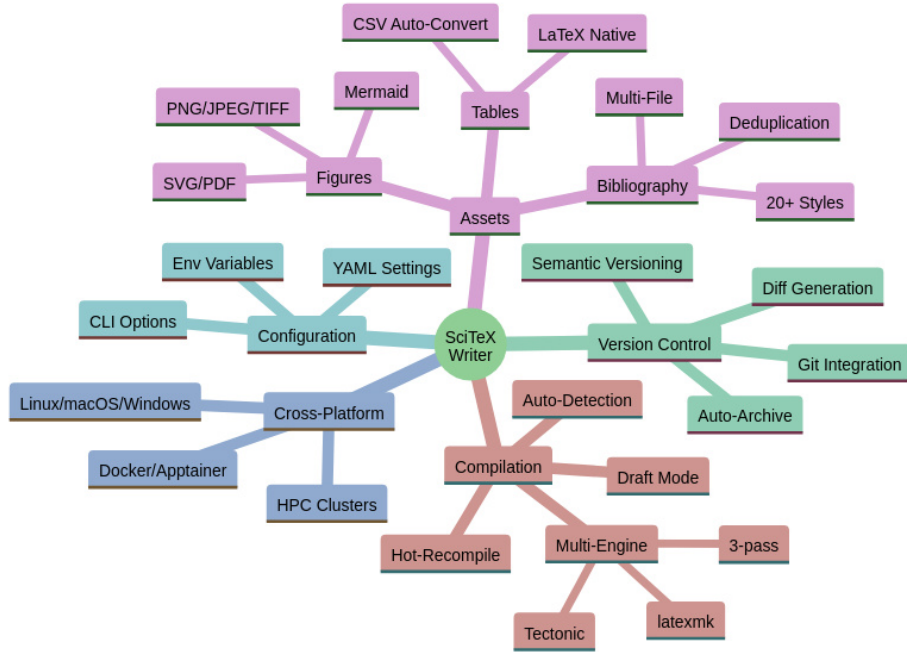


Figure 7 – SciTeX Writer Feature Overview. This mind map provides a comprehensive visualization of the framework’s major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.



Figure 8 – Multi-File Bibliography Processing and Deduplication.

The bibliography system enables researchers to organize references across multiple topical .bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).